

Master Sindico - Arquitetura de Solucoes & Plano de Execucao

Analise Arquitetural para Migracao de Monolito → Servicos Desacoplados

Autor: Analise tecnica consolidada **Data:** 23/03/2026 **Stack:** TypeScript/Go, Drizzle ORM, Bun, Zitadel (IAM), OpenSearch **Modelo de entrega:** Sprints de 14 dias **Equipe:** 1 full-stack/arquiteto + 1 dev mobile/web + 2 designers

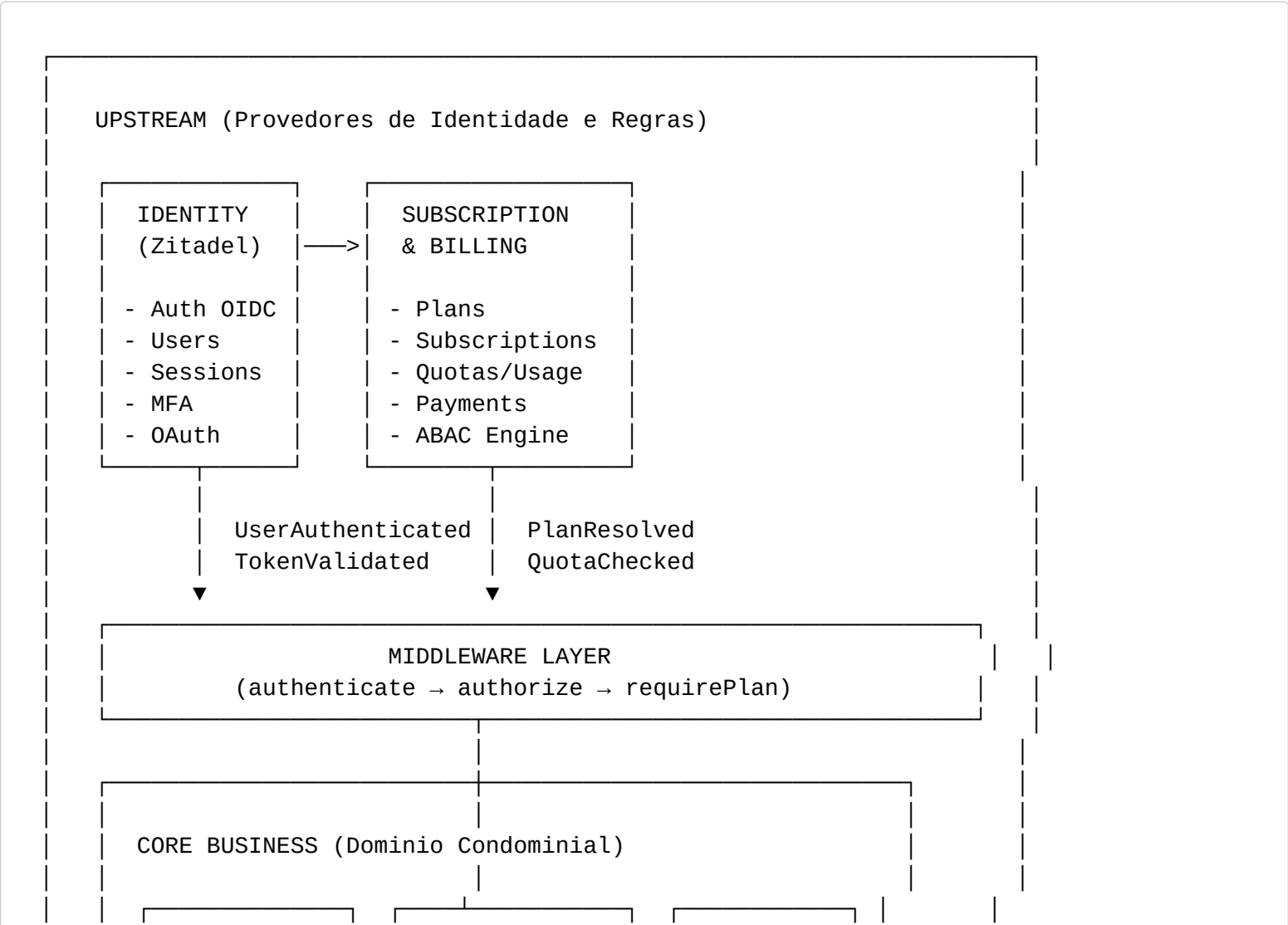
1. MAPEAMENTO DE DOMINIOS (Bounded Contexts)

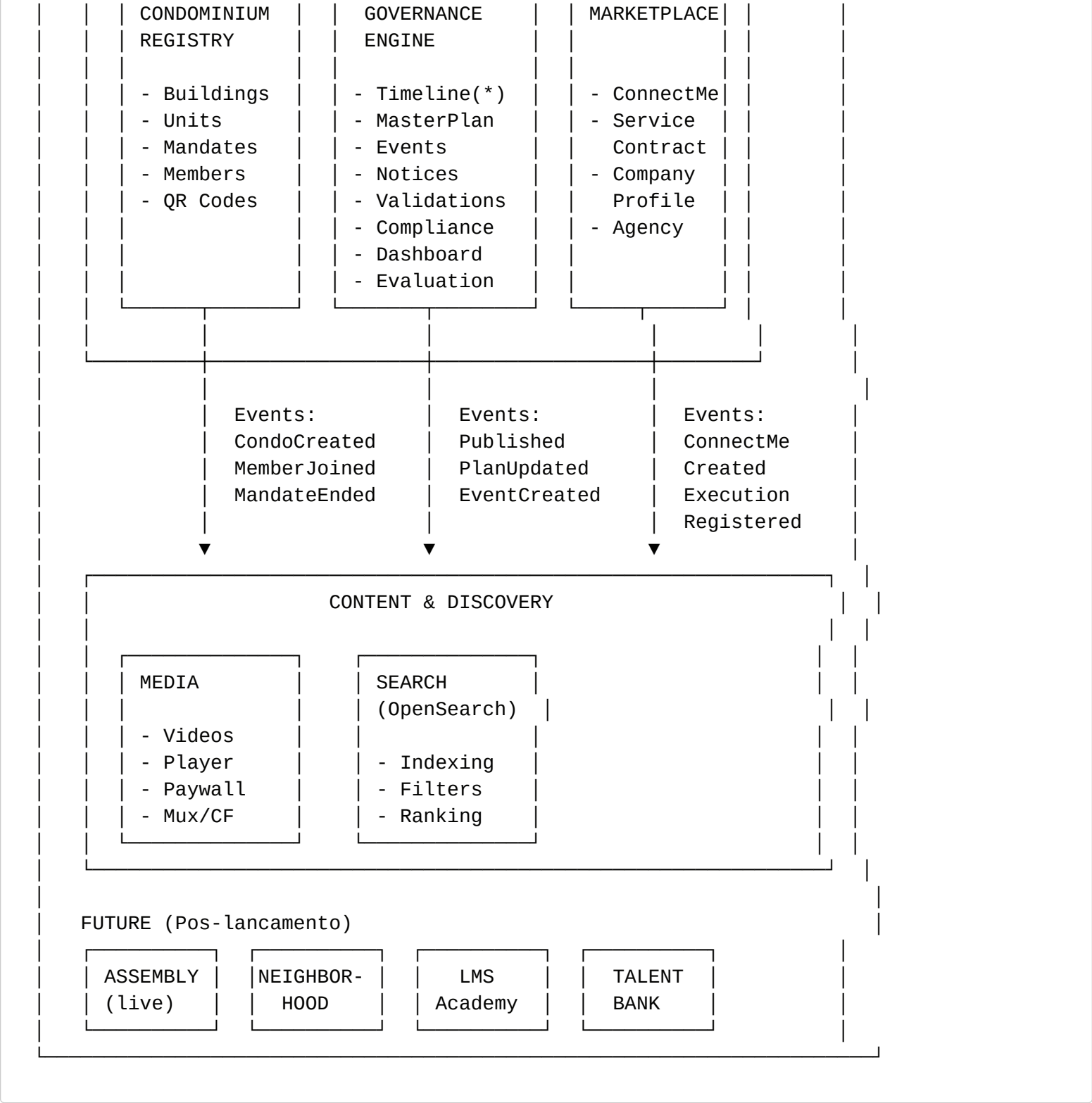
1.1 Principio de Separacao

Cada bounded context e um modulo autonomo que:

- Tem seu proprio schema de dados (tabelas isoladas)
- Expoe contratos publicos (DTOs, Events) e esconde internals
- Comunica com outros contextos via Domain Events (in-memory no monolito, message broker quando escalar)
- Tem seu proprio conjunto de use cases, entidades e repositorios

1.2 Mapa de Contextos





1.3 Tabela de Bounded Contexts

Context	Owner	DB Tables	Depende de	Publica Events
Identity	Zitadel (externo)	Zitadel DB	Nenhum	UserAuthenticated, UserCreated
Subscription	Nosso codigo	plans, subscriptions, transactions, invoices, payment_methods, permissions, plan_permissions, plan_quotas, feature_usages	Identity	PlanResolved, SubscriptionExpired, QuotaExceeded

Context	Owner	DB Tables	Depende de	Publica Events
Condominium	Nosso codigo	condominiums, units, members, mandates, syndic_companies	Identity, Subscription	CondominiumCreated, MemberJoined, MandateEnded
Governance	Nosso codigo	timeline_entries, master_plan_actions, events, notices, validations, evaluations, compliance_records, audit_logs	Condominium, Subscription	TimelinePublished, PlanActionUpdated, EventCreated, NoticePublished
Marketplace	Nosso codigo	connect_me_requests, service_executions, technical_activities, technical_notices, company_profiles, company_users, agency_links	Identity, Subscription, Condominium	ConnectMeCreated, ExecutionRegistered, ExecutionValidated
Media	Nosso codigo	videos, video_views, video_likes, video_comments	Identity, Subscription	VideoUploaded, VideoViewed
Search	OpenSearch (externo)	OpenSearch indices	Todos (consume events)	Nenhum (read-only)

1.4 Regras de Fronteira

Regra 1: Nenhum contexto faz query direta em tabelas de outro contexto.

- Errado: Governance faz `SELECT * FROM subscriptions WHERE user_id = ?`
- Certo: Governance chama `SubscriptionService.canUserPublish(userId)` ou recebe `PlanResolved` via middleware

Regra 2: Comunicacao cross-context e via:

- Fase 1 (Monolito): Function calls via DI + In-memory EventBus
- Fase 2 (Scale): HTTP/gRPC + Message Broker (RabbitMQ/SQS)

Regra 3: Cada contexto tem seu proprio modulo Fastify:

```
// modules/governance/governance.module.ts
export function governanceModule(app: FastifyInstance) {
  // Registra DI, rotas, event handlers
  // NAO importa nada de modules/marketplace/
}
```

2. MATRIZ DE COMPLEXIDADE (Esforco vs Valor)

2.1 Quadrante de Prioridade

		VALOR DE NEGOCIO	
		Baixo	Alto
Esforço	Alto	EVITAR	STRATEGIC (investir)
	Baixo	BACKLOG	QUICK WINS (fazer logo)
Esforço	Alto	Assembly ao vivo LMS completo	Governance Eng Billing+ABAC Connect Me Zitadel setup
	Baixo	i18n Dark mode BI reports	Condo Registry Eventos CRUD Comunicados Avaliacao CEP Lookup

2.2 Detalhamento por Feature

Feature	Esforço (dias)	Valor	Quadrante	Sprint
Zitadel OIDC integration	5-7	Critico	STRATEGIC	S0
OpenSearch setup + sync	5-7	Critico	STRATEGIC	S0 (background)
Billing engine (Plans + Subscriptions + Payments)	8-10	Critico	STRATEGIC	S1
ABAC engine (CASL + Matriz Funcional)	5-7	Critico	STRATEGIC	S1
Onboarding 4 personas	3-4	Alto	STRATEGIC	S1
Condominium Registry CRUD	3-4	Alto	QUICK WIN	S2
Unit Registry + Membership	3-4	Alto	QUICK WIN	S2
Timeline engine (append-only)	7-10	Critico	STRATEGIC	S2
Master Plan (status automatico)	5-7	Critico	STRATEGIC	S2
Eventos CRUD	2-3	Alto	QUICK WIN	S3
Comunicados CRUD	2-3	Alto	QUICK WIN	S3
Validacoes Pendentes hub	3-4	Alto	QUICK WIN	S3
Dashboard Sindico	4-5	Alto	STRATEGIC	S3
Compliance (Termo + Auditoria)	3-4	Medio	QUICK WIN	S3
Avaliacao Gestao	2-3	Medio	QUICK WIN	S3
Connect Me (Sindico->Empresa)	5-7	Critico	STRATEGIC	S4

Feature	Esforço (dias)	Valor	Quadrante	Sprint
Service & Contract	5-7	Alto	STRATEGIC	S4
Company Profile + Users	3-4	Alto	QUICK WIN	S4
Video Library + Mux	5-7	Alto	STRATEGIC	S5
Player + Paywall	3-4	Alto	STRATEGIC	S5
Search (OpenSearch queries)	3-4	Alto	QUICK WIN	S5
Push Notifications (FCM)	2-3	Medio	QUICK WIN	S6
Email templates (Resend)	2-3	Medio	QUICK WIN	S6
PDF Generator	2-3	Medio	QUICK WIN	S6
Assembly assincrona	7-10	Alto	STRATEGIC	S7
Assembly ao vivo (WebSocket)	20-30	Alto	EVITAR (MVP)	Pos
LMS completo	15-20	Medio	EVITAR (MVP)	Pos
Vizinhanca completo	10-15	Medio	EVITAR (MVP)	Pos
Banco de Talentos	8-10	Baixo	EVITAR (MVP)	Pos

2.3 Regra de Ouro

Se uma feature leva mais de 10 dias de esforço E não bloqueia monetização, ela sai do MVP.

Aplicando essa regra:

- Assembly ao vivo → **OUT** (20-30 dias, não bloqueia monetização)
- LMS → **OUT** (15-20 dias, nice-to-have)
- Vizinhanca → **OUT** (10-15 dias, pode lançar depois)
- Banco de Talentos → **OUT** (8-10 dias, valor baixo)

3. MODELO DE SPRINT (14 dias) — VERTICAL SLICES

3.1 Recomendação: Vertical Slices (não Backend-then-Frontend)

Por que Vertical Slices para dev solo:

Critério	Backend → Frontend	Vertical Slices
Feedback loop	Lento (só vê resultado na semana 2)	Rápido (feature funcional em 2-3 dias)
Risco de integração	Alto (backend e frontend podem divergir)	Baixo (integra continuamente)
Motivação	Cai na semana 1 (só backend abstrato)	Constante (vê progresso real)
Demonstrável	Só no fim do sprint	A cada slice
Bug discovery	Tardio (semana 2)	Imediato
Paralelismo com Dev 2	Difícil (ambos no mesmo layer)	Fácil (cada um em slice diferente)

3.2 Anatomia de um Sprint de 14 dias

DIA 1-2: PLANNING & SCHEMA

- └─ Definir slices do sprint (3-5 slices)
- └─ Database migrations (Drizzle)
- └─ Zod schemas (packages/schemas)
- └─ Interfaces de repositório
- └─ Testes de schema (se necessário)

DIA 3-10: VERTICAL SLICES (2-3 dias cada)

- └─ Slice 1: [Entity + UseCase + Route + Tela] ← Feature A
- └─ Slice 2: [Entity + UseCase + Route + Tela] ← Feature B
- └─ Slice 3: [Entity + UseCase + Route + Tela] ← Feature C
- └─ (Dev 2 trabalha em slices paralelos no mobile/web)

DIA 11-12: INTEGRACAO & TESTES

- └─ Integration tests (fluxos E2E)
- └─ Bug fixes de integracao
- └─ Code review cruzado com Dev 2
- └─ Performance check (queries lentas?)

DIA 13-14: POLISH & DEPLOY

- └─ UI polish (com design final se disponível)
- └─ Error handling edge cases
- └─ Deploy para staging
- └─ Demo/review com stakeholders
- └─ Planning próximo sprint

3.3 Exemplo Concreto: Sprint 2 (Institutional I)

SPRINT 2: CONDOMINIUM + GOVERNANCE BASE (14 dias)

Slice 1 (dias 3-5): "Sindico cadastra condominio"

- └─ Backend: Condominium entity, CreateCondominiumUseCase, route POST /condominiums
- └─ Frontend: Tela de cadastro, form com Zod validation
- └─ Teste: Criar condominio → verificar no DB → ver na listagem
- └─ DONE: Feature funcional end-to-end

Slice 2 (dias 5-7): "Sindico registra atividade na Timeline"

- └─ Backend: TimelineEntry entity (append-only), CreateTimelineEntryUseCase
- └─ Frontend: Form de atividade (26 tipos), listagem com filtros
- └─ Teste: Criar atividade → aparece na timeline → filtros funcionam
- └─ DONE: Timeline basica funcional

Slice 3 (dias 7-9): "Plano Diretor reage a Timeline"

- └─ Backend: MasterPlanAction entity, event handler TimelinePublished→UpdatePlanStatus
- └─ Frontend: Listagem de acoes, status visual, vincular atividade ao PD
- └─ Teste: Publicar na timeline → status do PD muda automaticamente
- └─ DONE: Event-driven status funcional

Slice 4 (dias 9-10): "Morador vincula a condominio"

- └ Backend: Unit/Member entities, JoinCondominiumUseCase
- └ Frontend: Busca por ID, cadastro unidade, termo aceite
- └ Teste: Morador busca condo → cadastra unidade → aparece como membro
- └ DONE: Membership funcional

Dias 11-14: Integracao, testes, polish, deploy staging

3.4 Template de Vertical Slice

Cada slice segue este checklist:

```
## Slice: [Nome da feature]
### Backend
- [ ] Entity/Domain model
- [ ] Repository interface + Drizzle implementation
- [ ] Use Case(s)
- [ ] Route(s) + Zod request/response schemas
- [ ] Domain Events (se necessario)
- [ ] Authorization check (ABAC)

### Frontend
- [ ] Route/Page
- [ ] Form/UI component
- [ ] API integration (TanStack Query)
- [ ] Loading/Error states
- [ ] Validation (Zod shared schema)

### Verificacao
- [ ] Happy path funciona E2E
- [ ] Error handling testado
- [ ] Authorization testada (user sem permissao recebe 403)
```

4. ESTRATEGIA DE PARALELISMO

4.1 Modelo: Main Track + Background Track

A ideia e ter SEMPRE duas "faixas" rodando ao mesmo tempo, sem competir por atencao:

SEMANA TIPICA:

MAIN TRACK (80% do tempo)
Features de negocio via Vertical Slices
Alta concentracao, codigo novo, domain logic
BACKGROUND TRACK (20% do tempo)
Infra pesada rodando em paralelo
Baixa concentracao, config/setup, scripts automatizados

4.2 Como funciona na pratica

Exemplo: OpenSearch migration em paralelo com Sprint 2

SPRINT 2 - MAIN TRACK: Governance features (vertical slices)

|

Dia 1 (manha): Configurar OpenSearch docker-compose + index mappings

Dia 1 (tarde): Começar Slice 1 (Condominium Registry)

...

Dia 5: Escrever sync job (PostgreSQL → OpenSearch) como background task

...

Dia 10: Testar queries OpenSearch em paralelo com PostgreSQL

Dia 11-12: Integrar OpenSearch no Search module (swap provider)

|

RESULTADO: No fim do sprint, temos Governance + Search funcionando

4.3 Regras de Paralelismo

Regra	Descricao
Never block the main track	Se infra nao esta pronta, use mock/fallback e continue
Timebox background work	Max 2h/dia em background track. Mais que isso, promova pra main.
Background = automatable	Se requer pensamento profundo, nao e background, e main track
Feature flags	Background track sempre atras de feature flag ate estar pronta

4.4 Background Track por Sprint

Sprint	Main Track	Background Track
S0	Monorepo + Zitadel	OpenSearch cluster + Docker compose
S1	Billing + ABAC	OpenSearch index mappings + sync job
S2	Governance base	OpenSearch queries + search UI
S3	Governance completa	Mux video pipeline setup
S4	Marketplace	Email templates (Resend)
S5	Content + Search	Push notification setup (FCM)
S6	Mobile polish	PDF generator templates
S7	Assembly + polish	Monitoring (OpenTelemetry)

4.5 Paralelismo com Dev 2

VINNI (Full-stack/Arquiteto)

DEV 2 (Mobile/Web)

Sprint 0-1: Backend + Infra

Sprint 2-3: Backend + Frontend

Flutter setup + Auth OIDC

Flutter: telas Morador

→

→

Sprint 4-5: Backend + Frontend → Flutter: telas Empresa + Video
Sprint 6: Integracoes → Flutter: paridade completa
Sprint 7: Assembly + Polish → Flutter: Assembly + QA

REGRA: Vinni sempre 1 sprint a frente no backend.
Dev 2 consome APIs ja prontas.
Schemas compartilhados (packages/schemas) sao o contrato.

5. ORDEM DE IMPLEMENTACAO

5.1 Sequencia Logica com Dependencias

FASE 0: FUNDACAO (Sprint 0) – "Nada funciona sem isso"

- |
- ├ [1] Monorepo (Turborepo + Bun)
- ├ [2] PostgreSQL + Drizzle + Migrations base
- ├ [3] Redis
- ├ [4] Zitadel (OIDC) ← PRIORIDADE #1 (desafoga auth custom)
- ├ [5] OpenSearch cluster ← PRIORIDADE #2 (background track)
- ├ [6] Core contracts (BaseUseCase, DomainError, UnitOfWork)
- ├ [7] Fastify plugins (CORS, Helmet, Rate-limit, Swagger)
- └ [8] CI/CD pipeline (lint, typecheck, build)

FASE 1: IDENTIDADE & DINHEIRO (Sprint 1) – "Quem e voce e quanto paga"

- |
- ├ [9] Zitadel ↔ Fastify middleware (token validation, user sync)
- ├ [10] Plans + Subscriptions (Drizzle schema)
- ├ [11] Payment integration (Stripe/MP via Adapter)
- ├ [12] ABAC engine (CASL + Matriz Funcional hardcoded)
- ├ [13] Quotas + Feature Usage tracking
- ├ [14] Onboarding flows (4 personas)
- └ [15] Trial module (duracoes por persona)

FASE 2: CORE BUSINESS (Sprints 2-3) – "O produto de verdade"

- |
- ├ [16] Condominium Registry
- ├ [17] Unit Registry + Membership
- ├ [18] Timeline engine (append-only, imutavel)
- ├ [19] Master Plan (status reativo via events)
- ├ [20] Events + Communications + Validations
- ├ [21] Evaluation + Compliance (Termo, Auditoria)
- ├ [22] Dashboard Sindico (indicadores)
- └ [23] Painel Morador (leitura)

FASE 3: MARKETPLACE (Sprint 4) – "Sindicos encontram empresas"

- |
- ├ [24] Connect Me (Sindico→Empresa, quotas, LGPD)
- ├ [25] Service & Contract (Registro Execucao)
- ├ [26] Company Profile + User Management
- └ [27] Status de Envios hub

FASE 4: CONTEUDO (Sprint 5) – "Videos e descoberta"

- └─ [28] Video Library + Mux upload
- └─ [29] Player + Paywall (25% preview)
- └─ [30] OpenSearch queries (swap search provider)
- └─ [31] Content Discovery

FASE 5: INTEGRACOES (Sprint 6) – "Tudo conectado"

- └─ [32] Push Notifications (FCM)
- └─ [33] Email templates (Resend)
- └─ [34] CEP Lookup (ViaCEP)
- └─ [35] PDF Generator
- └─ [36] Mandato (encerrar/transferir)
- └─ [37] Empresa Sindica Vinculada

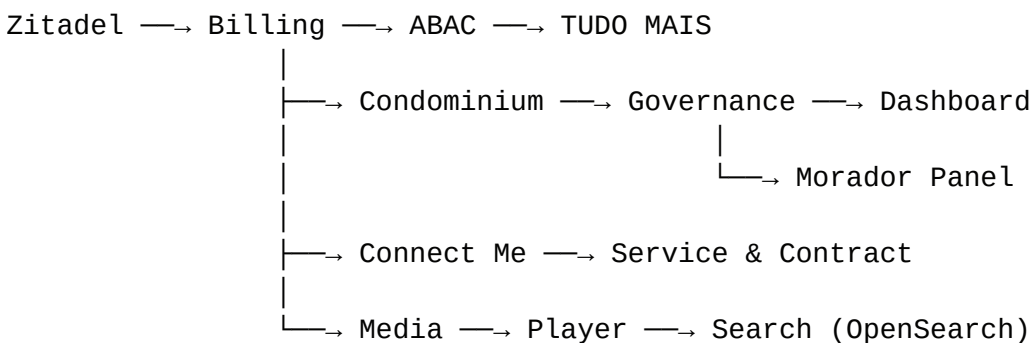
FASE 6: ASSEMBLEIA + POLISH (Sprint 7) – "Feature premium simplificada"

- └─ [38] Assembly assincrona (CRUD, pauta, votacao)
- └─ [39] Relatorios basicos
- └─ [40] Bug fixes acumulados
- └─ [41] Performance tuning

FASE 7: QA (Sprint 8/Buffer) – "Nao lanca quebrado"

- └─ [42] Testes E2E
- └─ [43] Security review
- └─ [44] Deploy production
- └─ [45] Monitoring (OpenTelemetry)
- └─ [46] Handoff docs

5.2 Grafo de Dependencias Criticas



Caminho critico: Zitadel → Billing → ABAC → Governance → Dashboard Se qualquer um desses atrasar, TUDO atrasa.

5.3 O Papel de Go

Go entra como opcao para servicos de alta performance que podem ser extraidos no futuro:

Servico	Linguagem	Motivo
API principal (Fastify)	TypeScript	Produtividade, type-safety, ecosystem

Servico	Linguagem	Motivo
Sync jobs (PostgreSQL → OpenSearch)	Go ou TS	Performance de batch processing
Video processing pipeline	Go	CPU-bound, concorrência nativa
Event bus worker (futuro)	Go	Throughput alto, memory footprint baixo
Assembly real-time (futuro)	Go	WebSocket scaling, goroutines

Recomendacao para MVP: Tudo em TypeScript. Go entra pos-lancamento para extrair servicos que precisam de performance.

6. CALENDARIO FINAL (8 Sprints)

Sprint	Periodo	Foco	Dev 1 (Vinni)	Dev 2	Background
S0	23/03→06/04	Fundacao	Monorepo, Zitadel, DB, Redis, Core	Flutter setup, OIDC	OpenSearch cluster
S1	07/04→20/04	Identity	Billing, ABAC, Onboarding, Trial	Flutter auth + onboarding	OS index mappings
S2	21/04→04/05	Institutional I	Registry, Timeline, Master Plan	Flutter home + timeline	OS sync job
S3	05/05→18/05	Institutional II	Events, Notices, Dashboard, Compliance	Flutter morador panel	Mux video pipeline
S4	19/05→01/06	Commercial	Connect Me, Service, Company Profile	Flutter empresa	Email templates
S5	02/06→15/06	Content	Video+Mux, Player, OpenSearch queries	Flutter video + search	FCM setup
S6	16/06→29/06	Integracoes	Push, Email, PDF, Mandato, ESV	Flutter paridade	PDF templates
S7	30/06→13/07	Assembly	Assembly assincrona, bugs, performance	Flutter assembly	OpenTelemetry
Buffer	14/07→08/08	QA	Testes E2E, security, deploy prod	Flutter QA	Monitoring

Marco 20/07: 80-90% pronto **Marco 08/08:** Entrega final

7. ESTRUTURA WIKI OBSIDIAN

Para ter toda a visao do projeto organizada, criar no vault `Clients/MasterSindico/`:

- 00 - Visao Geral/

00 - Indice.md01 - Produto.md

Portal de navegacao# O que e, problema, solucao

02 - Glossario.md	# Termos tecnicos/negocio
03 - Decisoes Pendentes.md	# Bloqueadores ativos
01 - Stack e Infra/	
01 - Stack Confirmada.md	# Tabela completa com versoes
02 - Zitadel.md	# Config, OIDC, roles, endpoints
03 - OpenSearch.md	# Indices, mappings, sync strategy
04 - Database Schema.md	# ERD, tabelas por contexto
05 - Design Tokens.md	# Cores OKLCH, tipografia
06 - Monorepo.md	# Turborepo, workspaces, scripts
07 - CI-CD.md	# Pipeline, deploy, environments
08 - Environments.md	# .env vars, secrets
02 - Dominios/	
00 - Domain Map.md	# Diagrama de bounded contexts
01 - Identity.md	# Zitadel integration spec
02 - Subscription.md	# Plans, billing, ABAC
03 - Condominium.md	# Registry, membership
04 - Governance.md	# Timeline, PD, events, compliance
05 - Marketplace.md	# Connect Me, service, company
06 - Media.md	# Videos, player, paywall
07 - Search.md	# OpenSearch queries, indexing
08 - Enums.md	# 36+ listas normalizadas
03 - Sprints/	
00 - Escopo MVP vs Futuro.md	# O que entra vs o que nao
01 - Sprint 0 Setup.md	# Checklist detalhado
02 - Sprint 1 Identity.md	
03 - Sprint 2 Institutional I.md	
04 - Sprint 3 Institutional II.md	
05 - Sprint 4 Commercial.md	
06 - Sprint 5 Content.md	
07 - Sprint 6 Integrations.md	
08 - Sprint 7 Assembly.md	
09 - Sprint 8 QA.md	
10 - Cronograma.md	# Timeline visual com datas
04 - Jornadas/	
01 - Sindico.md	# 31 telas, quais no MVP
02 - Morador.md	# 15 telas
03 - Empresa.md	# 16 telas
04 - Agencia.md	# Pos-lancamento
05 - Parceiro.md	# Pos-lancamento
05 - Entregas/	
(1 nota por marco com frontmatter date)	
06 - ADRs/	
ADR-001 Zitadel IAM.md	
ADR-002 OpenSearch.md	
ADR-003 Modular Monolith.md	
ADR-004 Vertical Slices.md	
ADR-005 Assembly Async MVP.md	
ADR-006 Payment Gateway.md	# PENDENTE

07 - Pos-Lancamento/	
01 - Assembly Live.md	
02 - LMS Academy.md	
03 - Vizinhanca.md	
04 - Banco Talentos.md	
05 - Compliance Avancado.md	
08 - Referencias/	
Escopo Antigo.md	
Requirements Kiro.md	# Link para .kiro/specs/

8. DECISOES PENDENTES (Resolver antes de Sprint 0)

#	Decisao	Opcoes	Impacto
1	Payment Gateway	Stripe vs Mercado Pago	Adapter Layer isola, mas PIX/Boleto sao diferenciais
2	Paywall	Hard block (403) vs Soft block (warning + grace)	Simplifica se hard block
3	Zitadel hosting	Cloud (managed) vs Self-hosted (Docker)	Cloud para MVP = menos ops
4	Go no MVP?	TypeScript only vs TS + Go para sync jobs	TS only simplifica, Go pos-lancamento
5	Mobile MVP scope	Core-only (auth, timeline, eventos) vs Full parity	Core-only nao bloqueia Dev 2

ARQUIVOS CRITICOS

Arquivo	Caminho
Escopo antigo	Content/excopo-tecnico-antigo.txt
Arquitetura	Content/contents/master-sindico-arquitetura-solucoes.md
Domain mapping	Content/contents/master-sindico-domain-mapping.md
Requirements	Content/.kiro/specs/master-sindico-platform/requirements.md
Codebase anterior	Development/full-stack/ (29K LOC referencia)
Obsidian Index	Vault: Clients/MasterSindico/00 - Visao Geral/00 - Indice Master Sindico.md